

# Máquina Virtual Java Simplificada (JVM)

Este projeto implementa uma Máquina Virtual Java simplificada, responsável por carregar arquivos `.class`, construir as estruturas necessárias para execução e interpretar um subconjunto de bytecodes Java.

Esta é a segunda fase do trabalho, sucedendo o Leitor/Exibidor de `.class`, que é utilizado aqui como módulo de carregamento de classes.

## 1. Objetivos da Fase 2

O objetivo desta etapa é implementar uma JVM funcional, porém reduzida, capaz de:

- Carregar uma classe `.class` utilizando o leitor desenvolvido na Fase 1.
- Construir as áreas de memória internas da JVM (Área de Métodos, Heap, Frames de Execução).
- Interpretar e executar instruções do bytecode Java.
- Criar e destruir frames de execução conforme chamadas de métodos.
- Executar o método `main` da classe fornecida.

Simplificações presentes:

- Não possui Garbage Collector.
- Não implementa o Verifier de bytecode.
- Não suporta múltiplas threads.
- Não possui suporte a JNI.
- Executa apenas um subconjunto essencial de instruções.

## 2. Arquitetura da JVM

A arquitetura foi dividida em módulos independentes, cada um responsável por uma parte do ambiente de execução.

## 2.1 Class Loader

Responsável por utilizar o Leitor/Exibidor da Fase 1 para:

- Ler o arquivo `.class`.
- Produzir a estrutura `ClassFile`.
- Registrar a classe na Área de Métodos.

O carregamento assume que o arquivo é válido.

## 2.2 Área de Métodos (Method Area)

A Área de Métodos funciona como o repositório de classes carregadas. Armazena:

- Metadados da classe;
- Métodos e seus atributos `Code`;
- Tabelas auxiliares necessárias para execução;
- Informações necessárias para resolução de métodos durante instruções como `invokestatic` e `invokevirtual`.

Cada classe carregada mantém ponteiros para sua estrutura `ClassFile` e para métodos acessíveis.

## 2.3 Heap

A Heap armazena:

- Objetos instanciados;
- Arrays de tipos primitivos ou referências.

Uso de funções do SO:

- `malloc()`
- `calloc()`
- `free()`

Sem Garbage Collector: a liberação ocorre apenas no final da execução da JVM.

Cada objeto contém:

- Referência para sua classe;
- Vetor de campos (instância);
- Informações específicas para arrays.

## 2.4 Frames de Execução

Cada chamada de método cria um Frame contendo:

- Variáveis locais;
- Pilha de operandos;
- Contador de programa (PC);
- Referência para o método em execução;
- Ponteiro para o frame anterior (modelando a pilha de chamadas).

Após o retorno, o frame é destruído e o valor retornado é movido para o frame anterior.

## 2.5 Engine de Execução (Interpretador)

A Engine é o laço principal da JVM, executando bytecodes sequencialmente.

O professor especificou que **não deve ser usado switch-case**; em vez disso, cada opcode referencia uma função via ponteiros armazenados em uma tabela de 256 posições.

Exemplo de inicialização:

```
OpcodeFunc tabela_opcodes[256];

void inicializar_tabela_opcodes() {
    for (int i = 0; i < 256; i++)
        tabela_opcodes[i] = NULL;

    tabela_opcodes[0x60] = op_iadd;    // iadd
    tabela_opcodes[0xAC] = op_ireturn; // ireturn
    tabela_opcodes[0xA7] = op_goto;    // goto
}
```

Loop principal da Engine:

```

while (jvm->frame_atual != NULL) {
    u1 opcode = frame->code[frame->pc];
    OpcodeFunc func = tabela_opcodes[opcode];

    if (func == NULL) {
        printf("Opcode não implementado: 0x%02X\n", opcode);
        exit(1);
    }

    func(jvm, frame);
}

```

## 3. Conjunto de Bytecodes Suportados

A implementação inclui um subconjunto mínimo:

### Aritmética

- iadd, isub, imul, idiv

### Load/Store

- iload, istore, aload, ldc, bipush

### Controle de fluxo

- goto, ifeq, ifne, return, ireturn

### Objetos

- new, getfield, putfield, invokestatic, invokevirtual

Novas instruções podem ser adicionadas apenas criando funções e registrando na tabela de opcodes.

## 5. Compilação e Execução

Para compilar:

```
make
```

Para executar:

```
./bin/jvm Teste.class
```

Pré-requisito: a classe deve ser compilada previamente com `javac` .

## 6. Testes Sugeridos

### Operações aritméticas

```
public class Teste {  
    public static void main(String[] args) {  
        int x = 2 + 3;  
    }  
}
```

### Métodos estáticos

```
class T {  
    static int soma(int a, int b) { return a + b; }  
  
    public static void main(String[] args) {  
        soma(1, 2);  
    }  
}
```

# Criação de objetos

```
class A { int x; }

public class Teste {
    public static void main(String[] args) {
        A a = new A();
        a.x = 10;
    }
}
```

## 7. Divisão do Trabalho (5 Integrantes)

| Integrante | Módulo                         | Responsabilidades                                                                                                                                         |
|------------|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1          | JVM base                       | Implementação de <code>jvm.c</code> , inicialização, finalização, integração com engine e área de métodos, ajustes em <code>main.c</code> .               |
| 2          | Class Loader e Área de Métodos | Integração com leitor da fase 1, armazenamento de classes, busca de métodos.                                                                              |
| 3          | Frames e Pilha                 | Implementação de <code>frame.c</code> , criação e destruição de frames, pilha de operandos, variáveis locais.                                             |
| 4          | Heap e Objetos                 | Implementação de objetos e arrays, alocação com <code>malloc/calloc</code> , <code>bytecodes new</code> , <code>getfield</code> , <code>putfield</code> . |
| 5          | Engine e Bytecodes             | Implementação da Engine com tabela de ponteiros, criação da tabela, <code>bytecodes</code> aritméticos, de controle e <code>load/store</code> .           |